SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO4 – ASSIGNMENT (PART 1)

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: CSA0601

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

TASK 1 – ALGORITHM DESIGN REVIEW

Diagnose and Repair Poor Pseudocode

The logistics company must load packages into a delivery vehicle with limited capacity. Each package has a weight and a value. Packages may be divided, so a fraction of a package can be loaded. The objective is to maximise the total value without exceeding the vehicle capacity.

This is the Fractional Knapsack Problem, for which a greedy strategy based on value per unit weight is appropriate.

➢ Task 1.1 – Diagnose the Design

The junior engineer proposes the following pseudocode:

```
ALGORITHM LoadVehicle

BEGIN
    Read packages
    Arrange packages
    Select the best package

    WHILE vehicle has space DO
        Add suitable package
    END WHILE

    Calculate value
    RETURN result
END
```

The following deficiencies are identified:

No.	Deficiency	Explanation
1	Incomplete input specification	The algorithm does not specify the package ID, weight, value, or vehicle capacity.
2	Undefined arrangement rule	It does not explain how packages should be arranged or sorted.
3	Ambiguous meaning of "best"	It does not define whether the best package means highest value or highest value-to-weight ratio.
4	Missing feasibility check	It does not check whether a package fits within the remaining vehicle capacity.
5	No fractional-package handling	It does not explain how to load part of a package when the complete package does not fit.
6	Remaining capacity is not updated	The algorithm must subtract the loaded weight from the available capacity.
7	Unclear termination condition	It must stop when the vehicle is full or all packages have been examined.
8	Incomplete result construction	It must record the selected packages, loaded fractions, and total value.

The proposed pseudocode is not sufficiently precise for implementation because it contains broad instructions but does not define the mathematical decisions and state updates required to solve the problem.

- **Task 1.2 – Repair the Design**

Greedy-choice rule: At each step, select the remaining package with the highest value-to-weight ratio. Load the whole package if it fits; otherwise, load only the fraction that fills the remaining capacity.

The value-to-weight ratio is calculated as:

$$\text{Ratio} = \frac{\text{Package Value}}{\text{Package Weight}}$$

Assumption: Package weights must be positive, package values must be non-negative, and vehicle capacity must be non-negative.

Corrected pseudocode:

```
ALGORITHM FractionalKnapsack(Packages, Capacity)
INPUT:
    Packages: collection of (PackageID, Weight, Value)
    Capacity: maximum weight that the vehicle can carry
OUTPUT:
    Selected package portions and MaximumTotalValue
BEGIN
    TotalValue ← 0
    RemainingCapacity ← Capacity
    SelectedPackages ← empty collection

    FOR each package P in Packages DO
        P.Ratio ← P.Value / P.Weight
    END FOR

    SORT Packages by Ratio in descending order

    FOR each package P in Packages DO
        IF RemainingCapacity = 0 THEN
            BREAK
        END IF

        IF P.Weight ≤ RemainingCapacity THEN
            Fraction ← 1
            LoadedWeight ← P.Weight
            LoadedValue ← P.Value
        ELSE
            Fraction ← RemainingCapacity / P.Weight
            LoadedWeight ← RemainingCapacity
            LoadedValue ← Fraction × P.Value
        END IF

        Add (P.PackageID, Fraction, LoadedWeight, LoadedValue)
            to SelectedPackages
        TotalValue ← TotalValue + LoadedValue
        RemainingCapacity ← RemainingCapacity - LoadedWeight
```

Explanation of the Repaired Algorithm

1. First, the algorithm calculates the value-to-weight ratio for every package.
2. The packages are sorted in descending order of this ratio.
3. The algorithm selects packages according to their ratios.
4. If a complete package fits, the entire package is loaded.
5. If it does not fit, only the required fraction is loaded.
6. The remaining capacity and total value are updated after every selection.
7. The algorithm stops when the vehicle becomes full or all packages have been considered.

➢ **Illustrative Trace**

Consider a vehicle with a capacity of 50 units.

Package	Weight	Value	Value / Weight
P1	10	60	6.0
P2	20	100	5.0
P3	30	120	4.0

Step-by-step execution:

Decision	Loaded Amount	Value Added	Remaining Capacity	Total Value
Start	0	0	50	0
Load P1 completely	10	60	40	60
Load P2 completely	20	100	20	160
Load 20/30 of P3	20	80	0	240

The vehicle uses all 50 units of capacity.

$$\text{Total Value} = 60 + 100 + 80 = \boxed{240}$$

➢ **Task 1.3 – Design Defence**

The repaired solution is pseudocode because it expresses a finite sequence of algorithmic operations in a structured, language-independent form. It identifies the input and output, defines the greedy selection criterion, specifies the feasibility check, describes the fractional selection calculation, and updates the remaining capacity. It does not use the syntax or library functions of a particular programming language. Each instruction is sufficiently precise for another person to implement without guessing what "best" or "suitable" means.

TASK 2 – FROM BUSINESS REQUIREMENT TO GREEDY PSEUDOCODE

Job Sequencing with Deadlines

A cloud-service company receives independent computational jobs. Every job requires one processing slot and has a deadline and a profit. A job earns its profit only if it is completed on or before its deadline. Since only one job can occupy a slot, the aim is to select and schedule jobs to maximise total profit.

➢ **Task 2.1 – Model the Requirement**

Element	Description
Input	A collection of jobs. Each job has a unique ID, deadline, and profit.
Output	A schedule assigning selected jobs to processing slots, together with total profit.
Objective	Maximise the sum of profits of all scheduled jobs.
Constraints	Each job takes exactly one slot; only one job can occupy a slot; a job must be completed no later than its deadline; each job can be scheduled at most once.
Decision	Decide whether a job can be accepted and, if so, which feasible slot should be assigned.
Required information	Profit determines processing priority, deadline determines the latest permissible slot, and slot occupancy determines feasibility.

➢ Task 2.2 – Formulate a Greedy Hypothesis

Greedy-choice rule: Process jobs in descending order of profit and assign each job to the latest unoccupied slot at or before its deadline.

This rule considers higher-profit jobs first. Placing a job as late as possible preserves earlier slots for jobs that may have earlier deadlines. For the standard unit-time job sequencing problem, this greedy strategy produces a maximum-profit feasible schedule.

➢ Task 2.3 – Design the Algorithm

```
ALGORITHM JobSequencing(Jobs)
INPUT:
    Jobs: collection of (JobID, Deadline, Profit)
OUTPUT:
    Schedule and TotalProfit
BEGIN
    IF Jobs is empty THEN
        RETURN empty schedule, 0
    END IF
```

```
    MaximumDeadline ← maximum Deadline among all jobs
    NumberOfSlots ← MaximumDeadline
    FOR slot ← 1 TO NumberOfSlots DO
        Schedule[slot] ← EMPTY
    END FOR

    SORT Jobs by Profit in descending order
    TotalProfit ← 0

    FOR each job J in Jobs DO
        Slot ← MIN(J.Deadline, NumberOfSlots)
        WHILE Slot ≥ 1 DO
            IF Schedule[Slot] = EMPTY THEN
                Schedule[Slot] ← J.JobID
                TotalProfit ← TotalProfit + J.Profit
                BREAK
            ELSE
                Slot ← Slot - 1
            END IF
        END WHILE
    END FOR

    RETURN Schedule, TotalProfit
END
```

The algorithm starts at the job's deadline and searches backwards. Placing a job in the latest feasible slot leaves earlier positions available for jobs that may have earlier deadlines.

➢ Task 2.4 – Desk-Check

Consider the following jobs:

Job	Deadline	Profit
A	2	80
B	1	35
C	2	60
D	3	40
E	1	25

Step 1: Sort jobs by descending profit.

Order	Job	Deadline	Profit
1	A	2	80
2	C	2	60
3	D	3	40
4	B	1	35
5	E	1	25

The maximum deadline is 3, so there are three available slots.

Initial schedule:

Slot 1	Slot 2	Slot 3
EMPTY	EMPTY	EMPTY

Step 2: Process each job.

Job	Slot Checks	Decision	Schedule After Decision	Total Profit
A	Slot 2 is empty	Place A in slot 2	[–, A, –]	80
C	Slot 2 occupied; slot 1 empty	Place C in slot 1	[C, A, –]	140
D	Slot 3 is empty	Place D in slot 3	[C, A, D]	180
B	Slot 1 occupied; no earlier slot	Reject B	[C, A, D]	180
E	Slot 1 occupied; no earlier slot	Reject E	[C, A, D]	180

Final Schedule:

Processing Slot	Job	Profit
1	C	60
2	A	80
3	D	40
Total		180

The selected jobs are C, A, and D.

$$\text{Maximum Total Profit} = 60 + 80 + 40 = \boxed{180}$$

➢ **Task 2.5 – Challenge Your Design**

Change one input: increase the profit of Job B from 35 to 100. All other values remain unchanged.

Job	Deadline	Modified Profit
A	2	80
B	1	100
C	2	60
D	3	40
E	1	25

Prediction: The schedule will change because B becomes the highest-profit job. Since its deadline is 1, it must be placed in slot 1 if selected.

Modified order:

B (100), A (80), C (60), D (40), E (25).

Modified schedule:

Job	Slot Decision	Resulting Schedule	Total Profit
B	Slot 1 is empty; place B	[B, –, –]	100
A	Slot 2 is empty; place A	[B, A, –]	180
C	Slots 2 and 1 are occupied; reject	[B, A, –]	180
D	Slot 3 is empty; place D	[B, A, D]	220
E	Slot 1 is occupied; reject	[B, A, D]	220

The prediction is correct. The changed profit causes B to be considered first and occupy slot 1. The new schedule is B, A, and D.

$$\text{New Total Profit} = 100 + 80 + 40 = \boxed{220}$$

The profit increases by 40 compared with the original schedule.

TASK 3 – FROM RECURRENCE TO EXECUTABLE LOGIC

Dynamic Programming – 0/1 Knapsack

In the 0/1 Knapsack problem, each item has a weight and a value. Items are indivisible: an item must either be selected completely or not selected. The total weight must not exceed the vehicle capacity, and the total value must be maximised.

➢ **Task 3.1 – Define the State**

Let DP[i][w] represent the maximum total value obtainable by considering only the first i items and using a knapsack capacity of at most w.

- i is the number of items considered, from 0 to n.
- w is the capacity currently available, from 0 to the given maximum capacity.
- DP[i][w] stores the best value achievable under those two limits.

The state does not merely represent the value of item i. It represents the optimum result among all valid combinations of the first i items that fit within capacity w.

➢ **Task 3.2 – Develop the Recurrence**

For item i, there are two possible decisions.

Case 1: The item cannot fit

If Weight[i] > w, the item cannot be included. The best value is therefore the same as the best value obtained using the first i − 1 items.

$$DP[i][w] = DP[i-1][w]$$

Case 2: The item can fit

If Weight[i] ≤ w, compare excluding the item with including it.

- Exclude: $DP[i-1][w]$
- Include: $Value[i] + DP[i-1]\big[w - Weight[i]\big]$

The remaining capacity becomes $w - Weight[i]$, and only the first i − 1 items may be used in the remaining subproblem.

The recurrence is:

$$DP[i][w] = \max\big(DP[i-1][w], Value[i] + DP[i-1]\big[w - Weight[i]\big]\big)$$

Boundary conditions

$$DP[0][w] = 0$$

for every capacity w, because no items are available.

$$DP[i][0] = 0$$

for every item count i, because a zero-capacity knapsack cannot hold any positive-weight item.

➢ Task 3.3 – Translate the Recurrence into Pseudocode

```
ALGORITHM ZeroOneKnapsack(Items, Capacity)
INPUT:
    Items: collection of (ItemID, Weight, Value)
    Capacity: maximum permitted weight
OUTPUT:
    MaximumValue and SelectedItems
BEGIN
    n ← number of items
    CREATE DP[0...n][0...Capacity]

    FOR i ← 0 TO n DO
        DP[i][0] ← 0
    END FOR

    FOR w ← 0 TO Capacity DO
        DP[0][w] ← 0
    END FOR

    FOR i ← 1 TO n DO
        FOR w ← 1 TO Capacity DO
            IF Weight[i] ≤ w THEN
                ExcludeValue ← DP[i - 1][w]
                IncludeValue ← Value[i] + DP[i - 1][w - Weight[i]]
                DP[i][w] ← MAX(ExcludeValue, IncludeValue)
            ELSE
                DP[i][w] ← DP[i - 1][w]
            END IF
        END FOR
    END FOR

    MaximumValue ← DP[n][Capacity]
    SelectedItems ← empty collection
    i ← n
    w ← Capacity

    WHILE i > 0 AND w > 0 DO
        IF DP[i][w] > DP[i - 1][w] THEN
            Add ItemID[i] to SelectedItems
            w ← w - Weight[i]
        END IF
        i ← i - 1
    END WHILE

    Reverse SelectedItems
    RETURN MaximumValue, SelectedItems
END
```

The reconstruction uses a strict greater-than comparison. If including an item gives exactly the same value as excluding it, the algorithm may choose the exclusion path. This still returns an optimal solution. The DP table stores the optimum value, while the backward trace identifies one set of items achieving it.

➢ Task 3.4 – Desk-Check

Consider the following items:

Item	Weight	Value
P	2	12
Q	3	20
R	4	25
S	5	30

Vehicle capacity = 7

The DP table is filled row by row, where each row adds one more item to the set of items that may be considered.

Dynamic Programming Table

Items Considered / Capacity	0	1	2	3	4	5	6	7
No items	0	0	0	0	0	0	0	0
P	0	0	12	12	12	12	12	12
P, Q	0	0	12	20	20	32	32	32
P, Q, R	0	0	12	20	25	32	37	45
P, Q, R, S	0	0	12	20	25	32	37	45

Important State Calculations

For P (weight 2, value 12), at capacity 2 the item fits.

$$DP[1][2] = \max(0,12) = 12$$

The same value remains optimal for capacities 2 through 7.

For Q (weight 3, value 20), at capacity 5:

- Include value = 20 + DP[1][2] = 20 + 12 = 32.
- Exclude value = DP[1][5] = 12.

Therefore:

$$DP[2][5] = 32$$

This represents selecting P and Q.

For R (weight 4, value 25), at capacity 7:

- Include value = 25 + DP[2][3] = 25 + 20 = 45.
- Exclude value = DP[2][7] = 32.

Therefore:

$$DP[3][7] = 45$$

For S (weight 5, value 30), at capacity 7:

- Include value = 30 + DP[3][2] = 30 + 12 = 42.
- Exclude value = DP[3][7] = 45.

Therefore:

$$DP[4][7] = 45$$

Reconstructing the Selected Items

1. Start at DP[4][7] = 45. Since DP[4][7] = DP[3][7], item S is excluded.
2. Move to DP[3][7]. Since DP[3][7] = 45 is greater than DP[2][7] = 32, item R is selected. Remaining capacity becomes $7 - 4 = 3$.
3. Move to DP[2][3]. Since DP[2][3] = 20 is greater than DP[1][3] = 12, item Q is selected. Remaining capacity becomes $3 - 3 = 0$.

The selected items are Q and R.

Selected Item	Weight	Value
Q	3	20
R	4	25
Total	7	45

Thus, the maximum obtainable value is:

45

➢ **Task 3.5 – Explain the Dependency**

A DP state depends on smaller states that must already have been computed. DP[i][w] depends on DP[i − 1][w] when the current item is excluded and on DP[i − 1][w − Weight[i]] when it is included. Both states belong to the previous item row.

Processing items in increasing order from 1 to n ensures that the previous row is available before the current row is calculated. This dependency order prevents the algorithm from using unfinished results and ensures that each state follows the recurrence correctly.

TASK 4 – PSEUDOCODE FORENSICS

Find, Explain and Repair an Algorithmic Defect

The Word Break problem asks whether a text string can be divided completely into words that are present in a given dictionary. A dynamic programming array records whether each prefix of the text can be segmented.

➢ **Task 4.1 – Locate the Defect**

The exact defect is the condition inside the inner loop. The original pseudocode sets DP[i] to TRUE whenever substring(Text, j, i) is in the dictionary. It does not verify that the prefix before that substring can itself be segmented.

DP[j] must also be TRUE. Otherwise, a valid dictionary word could be found after an invalid prefix, and the algorithm would incorrectly conclude that the entire prefix is segmentable.

The complete logical requirement is:

$$DP[j] = TRUE \quad \text{AND} \quad substring(Text, j, i) \in Dictionary$$

➢ **Task 4.2 – Construct a Failing Input**

Text: "ab"

Dictionary: { "b" }

The correct result is FALSE because "a" is not a dictionary word and the complete string cannot be segmented.

i	j Tested	Substring	In Dictionary?	DP[j]	Faulty DP[i]
0	–	Empty prefix	–	–	TRUE
1	0	a	No	TRUE	FALSE
2	0	ab	No	TRUE	FALSE
2	1	b	Yes	FALSE	TRUE (incorrect)

At i = 2 and j = 1, the substring "b" is in the dictionary. The faulty algorithm therefore sets DP[2] to TRUE, even though DP[1] is FALSE. This is the first incorrect state.
The algorithm returns TRUE, which is wrong.

➢ **Task 4.3 – Repair the Algorithm**

Only the incomplete condition is changed. The structure of the original algorithm is retained.

```
ALGORITHM WordBreak(Text, Dictionary)
INPUT:
    Text: string of length n
    Dictionary: collection of valid words
OUTPUT:
    TRUE if Text can be completely segmented; otherwise FALSE
BEGIN
    DP[0] ← TRUE

    FOR i ← 1 TO n DO
        DP[i] ← FALSE
        FOR j ← 0 TO i - 1 DO
            IF DP[j] = TRUE AND substring(Text, j, i) is in Dictionary THEN
                DP[i] ← TRUE
                BREAK
            END IF
        END FOR
    END FOR

    RETURN DP[n]
END
```

The added DP[j] = TRUE condition ensures that the substring begins immediately after a prefix that has already been segmented successfully.

➢ **Task 4.4 – Regression Check**

Use a valid example that the original algorithm already handles correctly.
Text: "applepen"
Dictionary: { "apple", "pen" }

Prefix Length	Prefix	DP Value	Reason
0	Empty	TRUE	Base condition
1	a	FALSE	No valid segmentation
2	ap	FALSE	No valid segmentation
3	app	FALSE	No valid segmentation
4	appl	FALSE	No valid segmentation
5	apple	TRUE	The whole prefix is a dictionary word and DP[0] is TRUE
6	applep	FALSE	No valid segmentation ending at this position
7	applepe	FALSE	No valid segmentation ending at this position
8	applepen	TRUE	DP[5] is TRUE and "pen" is in the dictionary

The corrected algorithm returns TRUE and identifies the segmentation:

"apple" + "pen"

Therefore, the repair fixes the failing case while preserving correct behaviour for this valid case.

Time Complexity

For a text of length n, the algorithm considers $O(n^2)$ substrings in the straightforward implementation. Dictionary membership lookup time depends on the dictionary representation. With efficient set-based lookup, the usual DP analysis is $O(n^2)$ substring checks, excluding substring construction costs.

TASK 5 – SAME GRAPH, DIFFERENT ALGORITHMIC REASONING

Dijkstra, Bellman-Ford and Floyd-Warshall

The directed weighted graph contains the following vertices and edges.

Directed Edge	Weight
A → B	4
A → C	7
B → C	2
B → D	6
C → D	1
D → E	3
C → E	8

The source for the single-source shortest-path task is A.

➢ Task 5.1 – Greedy Shortest-Path Design

Dijkstra's algorithm is suitable because the assignment initially assumes that all edge weights are non-

negative. It repeatedly selects the unvisited vertex with the smallest tentative distance, then relaxes its outgoing edges.

Pseudocode:

```
ALGORITHM Dijkstra(Graph, Source)
INPUT:
    Directed weighted graph Graph = (V, E), with non-negative weights
    Source vertex
OUTPUT:
    Shortest distance and predecessor for each vertex
BEGIN
    FOR each vertex v in V DO
        Distance[v] ← INFINITY
        Predecessor[v] ← NULL
        Visited[v] ← FALSE
    END FOR

    Distance[Source] ← 0

    WHILE there exists an unvisited vertex DO
        Select unvisited vertex u with minimum Distance[u]
        IF Distance[u] = INFINITY THEN
            BREAK
        END IF

        Visited[u] ← TRUE

        FOR each edge (u, v) leaving u DO
            IF Visited[v] = FALSE THEN
                NewDistance ← Distance[u] + Weight(u, v)
                IF NewDistance < Distance[v] THEN
                    Distance[v] ← NewDistance
                    Predecessor[v] ← u
                END IF
            END IF
        END FOR
    END WHILE

    RETURN Distance, Predecessor
END
```

Dijkstra Manual Trace

Step	Selected Vertex	Relaxation Performed	Distance State (A, B, C, D, E)
Initial	–	Set A = 0; all others = ∞	(0, ∞, ∞, ∞, ∞)
1	A	A→B gives 4; A→C gives 7	(0, 4, 7, ∞, ∞)
2	B	B→C gives 6, improving 7; B→D gives 10	(0, 4, 6, 10, ∞)
3	C	C→D gives 7, improving 10; C→E gives 14	(0, 4, 6, 7, 14)
4	D	D→E gives 10, improving 14	(0, 4, 6, 7, 10)
5	E	No outgoing edges	(0, 4, 6, 7, 10)

Final Shortest Paths from A

Destination	Shortest Distance	Shortest Path
A	0	A
B	4	A → B
C	6	A → B → C
D	7	A → B → C → D
E	10	A → B → C → D → E

The shortest distance from A to E is: 10

➢ Task 5.2 – Change the Graph Assumption

If negative-weight edges are permitted, Dijkstra's greedy commitment can become unsafe. Once a vertex is marked visited, Dijkstra does not reconsider its distance. A negative edge encountered later may produce a shorter route to a vertex that was already finalised. The non-negative edge assumption is what makes the greedy finalisation valid.

Bellman-Ford Algorithm

Bellman-Ford handles negative-weight edges by repeatedly relaxing every edge. It does not permanently finalise a vertex after one minimum-distance selection.

```
ALGORITHM BellmanFord(Graph, Source)
INPUT:
    Directed weighted graph Graph = (V, E)
    Source vertex
OUTPUT:
    Shortest distances, predecessors, or negative-cycle indication
BEGIN
    FOR each vertex v in V DO
        Distance[v] ← INFINITY
        Predecessor[v] ← NULL
    END FOR
    Distance[Source] ← 0

    FOR pass ← 1 TO |V| - 1 DO
        Updated ← FALSE
        FOR each edge (u, v) with weight w in E DO
            IF Distance[u] ≠ INFINITY THEN
                IF Distance[u] + w < Distance[v] THEN
                    Distance[v] ← Distance[u] + w
                    Predecessor[v] ← u
                    Updated ← TRUE
                END IF
            END IF
        END FOR
        IF Updated = FALSE THEN
            BREAK
        END IF
    END FOR

    FOR each edge (u, v) with weight w in E DO
        IF Distance[u] ≠ INFINITY AND Distance[u] + w < Distance[v] THEN
            RETURN "Reachable negative-weight cycle exists"
        END IF
    END FOR

    RETURN Distance, Predecessor
END
```

A shortest simple path can contain at most |V| − 1 edges. Therefore, |V| − 1 complete passes are sufficient to propagate shortest distances when no reachable negative-weight cycle exists.

A further pass checks whether any edge can still reduce a distance. If so, a reachable negative-weight cycle exists, and no finite shortest distance is defined for vertices affected by that cycle.

➢ Task 5.3 – Change the Requirement

For all-pairs shortest paths, the organisation needs a shortest distance for every ordered pair of vertices, not just paths from one source.

Floyd-Warshall Algorithm uses Dynamic Programming to calculate these distances.

State Definition:

D[k][i][j] is the shortest distance from vertex i to vertex j when only the first k vertices in the chosen vertex ordering may be used as intermediate vertices.

Initialisation:

- $D[0][i][i] = 0$
- $D[0][i][j]$ equals the direct edge weight if edge i → j exists.
- Otherwise, the distance is INFINITY.

Transition:

For each possible intermediate vertex k, either the shortest route does not use k, or it uses k and is formed from a route i → k and a route k → j.

$$D[k][i][j] = \min(D[k-1][i][j], D[k-1][i][k] + D[k-1][k][j])$$

Pseudocode:

```
ALGORITHM FloydWarshall(Graph)
INPUT:
    Directed weighted graph Graph = (V, E)
OUTPUT:
    Shortest distance between every ordered pair of vertices
BEGIN
    FOR each vertex i in V DO
        FOR each vertex j in V DO
            IF i = j THEN
                Distance[i][j] ← 0
            ELSE IF edge i→j exists THEN
                Distance[i][j] ← Weight(i, j)
            ELSE
                Distance[i][j] ← INFINITY
            END IF
        END FOR
    END FOR

    FOR each vertex k in V DO
        FOR each vertex i in V DO
            FOR each vertex j in V DO
                IF Distance[i][k] ≠ INFINITY AND
                   Distance[k][j] ≠ INFINITY THEN
                    NewDistance ← Distance[i][k] + Distance[k][j]
                    IF NewDistance < Distance[i][j] THEN
                        Distance[i][j] ← NewDistance
                    END IF
                END IF
            END FOR
        END FOR
    END FOR

    RETURN Distance
END
```

Unlike Dijkstra and Bellman-Ford, which begin with one source and calculate distances from that source, Floyd-Warshall maintains a matrix of distances for every pair.

Its three nested loops systematically allow each vertex to become an intermediate vertex. A negative value on a diagonal Distance[i][i] after processing indicates a negative cycle.

TASK 6 – DESIGN CHALLENGE

Energy-Aware Job Scheduling in a Cloud Data Centre

The data centre has one energy-efficient server and a set of jobs. Each job has a start time, finish time and energy-saving value. Because the server can execute only one job at a time, selected jobs must have non-overlapping intervals. The goal is to maximise the sum of energy-saving values.

➢ Stage 1 – Interpret the Problem

Element	Interpretation
Input	A set of jobs, each containing Job ID, start time, finish time and energy-saving value.
Output	A compatible set of selected jobs and the maximum total energy-saving value.
Objective	Maximise the sum of values of selected jobs.
Constraints	A job is selected completely or not at all. Two selected jobs cannot overlap. If one job finishes at time t and another starts at time t, they are compatible.
Decisions	For each job, decide whether to include it or exclude it, considering the value of compatible earlier jobs.

This is a Weighted Interval Scheduling problem. The interval constraint means that selecting a job may prevent the selection of other jobs. Therefore, individual job value alone is not enough to make a correct global decision.

➢ Stage 2 – Propose the Strategy

Evaluation of Possible Strategies

Possible Strategy	Strength	Limitation
Earliest finishing time	Can maximise the number of mutually compatible jobs in the unweighted interval scheduling problem.	Does not maximise total energy-saving value when job values differ.
Highest individual value	Prioritises the job with the largest single benefit.	A high-value job may overlap several jobs whose combined value is greater.
Shortest duration	May leave more time available for other jobs.	Duration alone does not represent

		energy-saving benefit.
Consider compatible combinations	Compares the total value of including and excluding each job.	Requires storing and reusing subproblem results.

Selected Strategy: Dynamic Programming using Weighted Interval Scheduling.

This strategy is appropriate because the objective is a maximum total weight/value under interval compatibility constraints. For each job, the algorithm compares the best solution that excludes it with the best solution that includes it plus the best compatible earlier solution. This considers combinations and avoids committing to a locally attractive but globally poor job.

- **Stage 3 – Identify the Critical Information**

Sorting and Compatibility

Sort jobs in ascending order of finish time. For each job j, calculate p(j), the index of the last job before j in the sorted order whose finish time is less than or equal to the start time of j.

Compatibility condition:

FinishTime[i] ≤ StartTime[j]. If no earlier job satisfies this condition, p(j) =

State Definition

DP[j] represents the maximum total energy-saving value obtainable using only the first j jobs in finish-time order.

Decisions and Recurrence

- Exclude job j: the best value is DP[j − 1].
- Include job j: its value is added to the best solution among jobs compatible with it, giving Value[j] + DP[p(j)].

Therefore:

$$DP[j] = MAX(DP[j - 1], Value[j] + DP[p(j)])$$

Base Condition:

DP[0] = 0

because no jobs are available.

The dependency is valid because p(j) is less than j after sorting by finish time. Thus, DP[p(j)] and DP[j−1] have already been calculated when DP[j] is computed.

- **Stage 4 – Design Complete Pseudocode**

```
ALGORITHM EnergyAwareScheduling(Jobs)
INPUT:
    Jobs: collection of (JobID, StartTime, FinishTime, EnergySavingValue)
OUTPUT:
    SelectedJobs and MaximumEnergySavingValue
BEGIN
    SORT Jobs by FinishTime in ascending order
    n ← number of jobs

    CREATE P[1...n], DP[0...n], Take[1...n]

    FOR j ← 1 TO n DO
        P[j] ← 0
        FOR i ← j - 1 DOWNTO 1 DO
            IF FinishTime[i] ≤ StartTime[j] THEN
                P[j] ← i
                BREAK
            END IF
        END FOR
    END FOR

    DP[0] ← 0
    FOR j ← 1 TO n DO
        ExcludeValue ← DP[j - 1]
        IncludeValue ← EnergySavingValue[j] + DP[P[j]]

        IF IncludeValue > ExcludeValue THEN
            DP[j] ← IncludeValue
            Take[j] ← TRUE
        ELSE
            DP[j] ← ExcludeValue
            Take[j] ← FALSE
        END IF
    END FOR

    SelectedJobs ← empty collection
    j ← n
    WHILE j > 0 DO
        IF Take[j] = TRUE THEN
            Add JobID[j] to SelectedJobs
            j ← P[j]
        ELSE
            j ← j - 1
        END IF
    END WHILE

    Reverse SelectedJobs
    MaximumEnergySavingValue ← DP[n]
    RETURN SelectedJobs, MaximumEnergySavingValue
END
```

The Take array records the decision made for each DP state. During reconstruction, if Take[j] is TRUE, job j is included and the trace jumps to p(j). Otherwise, job j is excluded and the trace moves to j − 1. The reconstructed list is reversed to present jobs in chronological order.

Complexity Analysis

- Sorting takes O(nlog n).
- The straightforward compatibility search examines earlier jobs for each job and takes \(O(n^2)\) time in the worst case.
- Filling the DP table takes O(n).
- Reconstructing the selected jobs takes O(n).

Therefore, the overall time complexity of this version is:

$$\boxed{O(n^2)}$$

The auxiliary space complexity is:

$$\boxed{O(n)}$$

excluding the input and sorting storage.

➢ **Stage 5 – Desk-Check**

Given jobs:

Job	Start Time	Finish Time	Energy-Saving Value
J1	1	3	20
J2	2	5	50
J3	4	6	30
J4	6	8	40
J5	5	9	60
J6	8	10	30
J7	9	11	50
J8	11	13	40

The jobs are already in ascending order of finish time.

Step 1: Find the Last Compatible Job

Job	Start	Finish	Value	p(j) – Last Compatible Job
J1	1	3	20	0
J2	2	5	50	0
J3	4	6	30	J1
J4	6	8	40	J3
J5	5	9	60	J2
J6	8	10	30	J4
J7	9	11	50	J5
J8	11	13	40	J7

For example, J3 starts at time 4. J1 finishes at time 3, so J1 is compatible. J4 starts at time 6, and J3 finishes at time 6; equality is permitted by the assignment's assumption.

Step 2: Calculate the DP Values

Use:

$$DP[j] = \max(DP[j-1], Value[j] + DP[p(j)])$$

with:

$$DP[0] = 0$$

Job	p(j)	Exclude Value DP[j−1]	Include Value Value[j] + DP[p(j)]	DP[j]	Decision
J1	0	0	20 + 0 = 20	20	Include
J2	0	20	50 + 0 = 50	50	Include
J3	1	50	30 + 20 = 50	50	Tie; retain previous
J4	3	50	40 + 50 = 90	90	Include
J5	2	90	60 + 50 = 110	110	Include
J6	4	110	30 + 90 = 120	120	Include
J7	5	120	50 + 110 = 160	160	Include
J8	7	160	40 + 160 = 200	200	Include

The final optimum value is:

$$DP[8] = \boxed{200}$$

Step 3: Reconstruct the Selected Jobs

1. Start at j = 8. J8 is included; move to p(8) = 7.
2. J7 is included; move to p(7) = 5.
3. J5 is included; move to p(5) = 2.
4. J2 is included; move to p(2) = 0. Reconstruction stops.

Selected Job	Start	Finish	Value
J2	2	5	50
J5	5	9	60
J7	9	11	50
J8	11	13	40
Total			200

The selected jobs are J2, J5, J7 and J8. Their intervals are compatible: J2 ends at 5 when J5 starts; J5 ends at 9 when J7 starts; and J7 ends at 11 when J8 starts.

The maximum total energy-saving value is:

$$50 + 60 + 50 + 40 = \boxed{200}$$

➢ **Stage 6 – Attack Your Own Design**

1. Adversarial Dataset

Construct five jobs so that one job has a high individual value but overlaps several jobs whose combined value is greater.

Job	Start Time	Finish Time	Energy-Saving Value
A	1	11	100
B	1	4	45

C	4	7	45
D	7	10	45
E	10	12	20

2. Why This Dataset Is Challenging

Job A has the highest individual value (100), but it occupies the long interval from 1 to 11. It overlaps B, C and D. Those three compatible jobs together provide:

$$45 + 45 + 45 = 135$$

This is greater than A's value. Job E can also follow D because D finishes at 10 and E starts at 10.

The dataset therefore challenges a strategy that simply selects the highest-value job first.

3. Prediction

The prediction is that the dynamic programming algorithm will reject A and select B, C, D and E, producing a total value of 155.

4. Apply the Algorithm

After sorting by finish time, the order is B, C, D, A, E.

Job in Sorted Order	p(j)	Exclude	Include	DP Value	Decision
B	0	0	45	45	Include
C	B (1)	45	45 + 45 = 90	90	Include
D	C (2)	90	45 + 90 = 135	135	Include
A	0	135	100 + 0 = 100	135	Exclude
E	D (3)	135	20 + 135 = 155	155	Include

The key decision occurs at job A. Excluding A preserves the value 135 from B, C and D. Including A gives only 100. Therefore, A is excluded.

Job E can then be included after D, increasing the total to 155.

5. Final Result and Conclusion

Selected Job	Interval	Value
B	1–4	45
C	4–7	45
D	7–10	45
E	10–12	20
Total		155

The algorithm returns B, C, D and E with total energy-saving value 155. The prediction is correct. This is optimal: the alternative containing A cannot include B, C or D, and A also overlaps E. Selecting A alone gives 100. The compatible chain B, C, D and E gives 155.

The dynamic programming recurrence correctly compares include and exclude alternatives and is not defeated by the locally attractive job A.

The test confirms that the algorithm does not rely on the largest individual job value. It evaluates the best compatible combination and therefore addresses the global optimisation objective.

OVERALL CONCLUSION

This assignment demonstrates the design and analysis of algorithms for optimisation problems using Greedy strategies and Dynamic Programming.

Task 1 diagnoses incomplete pseudocode and repairs it using the Fractional Knapsack greedy-choice rule based on value-to-weight ratio. Task 2 develops a greedy job-sequencing algorithm that schedules high-profit jobs in the latest available feasible slots. Task 3 formulates the 0/1 Knapsack problem using a two-dimensional DP state, an include/exclude recurrence and a reconstruction procedure.

Task 4 identifies a logical defect in the Word Break algorithm and repairs it by checking whether the preceding prefix is segmentable. Task 5 compares Dijkstra, Bellman-Ford and Floyd-Warshall according to their assumptions and shortest-path requirements. Task 6 independently models Energy-Aware Job Scheduling as Weighted Interval Scheduling and solves it using Dynamic Programming.

The manual traces, modified inputs, regression test and adversarial dataset demonstrate why algorithm design must include more than a final numerical answer. A complete algorithm must define its inputs, outputs, objective, constraints, state or decision rule, processing order, termination condition and result reconstruction.

Overall, the assignment shows that selecting an appropriate algorithmic strategy depends on the structure of the problem. Greedy methods are suitable when a safe local choice can be established, while Dynamic Programming is appropriate when a global optimum can be constructed from overlapping subproblems and valid state dependencies.
